Institute of Technology of Cambodia
Department of Applied Mathematics and Statistics

Parallel and Distribution System
I4-AMS-DAS-B

DROUGHT PREDICTION

using GRU Neural Network

Lectured by:
Prof. LONG Pakrigna

Presented By:	**ID:**
NOV Panhavath	e20221643
Sophat Vitou	e20220813

TABLE OF CONTENTS

PROBLEM STATEMENT

Drought is one of the most damaging natural disasters but unlike floods or earthquakes, it builds silently over weeks.

The 3 core problems:

- Traditional drought monitoring is slow experts manually assess data weekly
- Drought develops gradually over many days, not overnight — hard to detect early
- 19+ million rows of historical weather data exists but is too large for standard tools

Can we use the past 7 days of weather data to automatically predict how severe drought will be tomorrow?

Project Goal: Build an end-to-end deep learning pipeline that predicts drought severity using Big Data tools (PySpark) and sequence models (GRU).

DATASET OVERVIEW

Data Source: **Kaggle**

	train_timeseries.csv	soil_data.csv
What it is	• Daily weather per county	• Fixed land info per county
Rows	• 19.3 million	• 3,109
Changes	• Every day	• Never changes
Key column	• fips + date	• fips

How they connect:

- fips = US county ID code, the key that links both files
- Left join on fips, keep all daily rows, attach soil info to each

After joining (left join that **train_timeseries.csv** as left table):

- Drop rows where score is NULL → can't train without a label
- Final usable rows: ~2.75 million

Dataset is too large for pandas, so i use PySpark to process it in parallel.

EDA & FEATURE SELECTION

Before training, i explored the data to understand patterns and pick only the most useful features.

EDA = Exploratory Data Analysis, looking at the data visually before building the model.

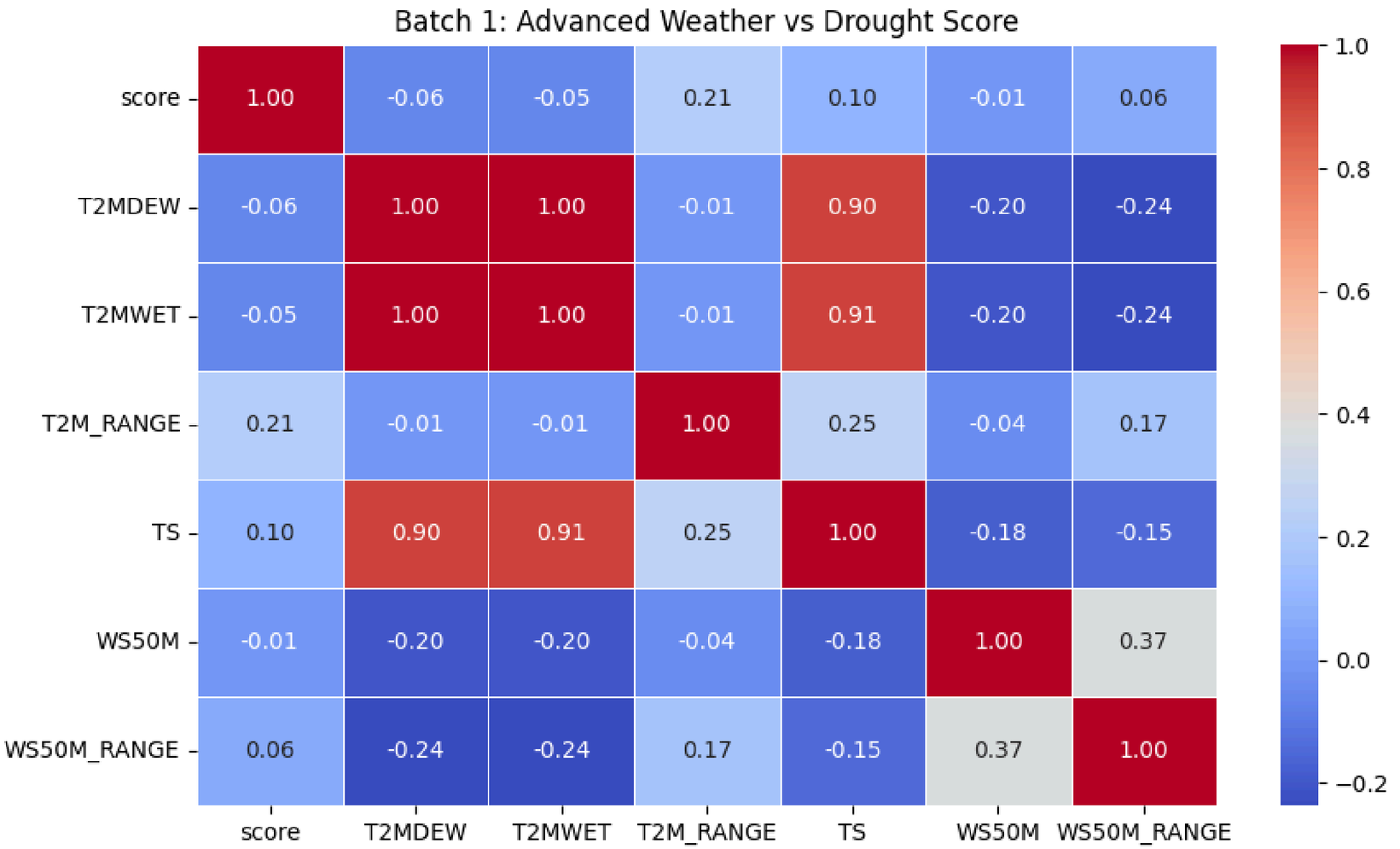
What	Why
Distribution of each feature	Check for skewed or abnormal values
Correlation between features and score	Find which features actually affect drought
Missing value analysis	Know which columns to fill or drop

Feature Selection result:
Started with all columns from both files after left join → used correlation heatmap → selected 18 most relevant features.

EDA & FEATURE SELECTION

Example choosing features based on heat map correlation

Since Features such as **T2MDEW**, **T2MWET**, **T2M_RANGE**, **TS**, **WS50M_RANGE** have correlation higher than or equal |0.05| so i decide to choosing them as features.



FEATURES & TARGET

After EDA and correlation analysis, i selected 18 features across 4 groups by choosing every single column that scores higher than 0.05 or lower than -0.05 against the score.

Weather

- PS: Air Pressure
- QV2M: Humidity
- T2M_MAX: Maximum Temperature
- T2MDEW: Dew Point
- T2MWET: Wet Bulb Temperature
- T2M_RANGE: Temperature Range
- TS: Ground Surface Temperature

Land Cover

- NVG_LAND: Bare Land (No Vegetation)
- GRS_LAND: Grassland
- FOR_LAND: Forest
- CULTRF_LAND: Rainfed Farmland
- CULTIR_LAND: Irrigated Farmland
- CULT_LAND: Total Farmland

Geography

- lat: Latitude
- lon: Longitude
- elevation: Elevation

Wind

- WS10M_RANGE: Wind Speed Variation (10 meters)
- WS50M_RANGE: Wind Speed Variation (50 meters)

FEATURES & TARGET

Target column is **score:**

Score	Meaning
0	No drought
1	Abnormally dry
2	Moderate drought
3	Severe drought
4	Extreme drought
5	Exceptional drought

- Target Variable: A 6-point scale (0 to 5).
- 0 = Normal, healthy conditions.
- 1 to 5 = Increasing levels of drought severity.

MODEL ARCHITECTURE

I use a GRU neural network, designed specifically for sequence and time-series data.

- GRU = Gated Recurrent Unit. It reads data day by day and remembers important patterns from previous days, like how a human notices "it hasn't rained for 7 days straight."

MODEL ARCHITECTURE

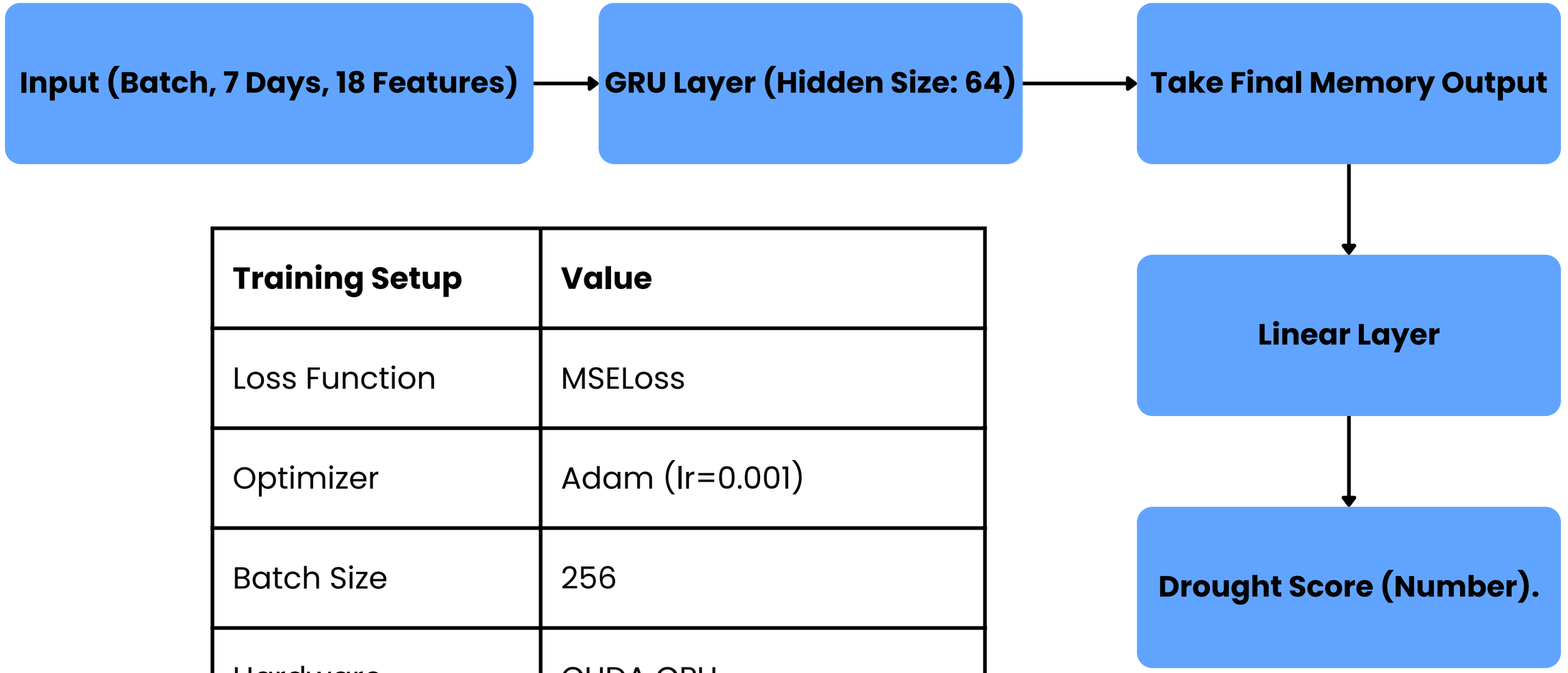
Why GRU not normal neural network:

Normal Neural Network	GRU
Looks at one snapshot	Reads a sequence of days
No memory	Has memory across time steps
Bad for time-series	Built for time-series

The Sliding Window concept (most important thing to explain):

Day 1 Day 2 Day 3 Day 4 Day 5 Day 6 Day 7 → Predict Day 8
Day 2 Day 3 Day 4 Day 5 Day 6 Day 7 Day 8 → Predict Day 9
Day 3 Day 4 Day 5 Day 6 Day 7 Day 8 Day 9 → Predict Day 10
...

MODEL ARCHITECTURE



Training Setup	Value
Loss Function	MSELoss
Optimizer	Adam (lr=0.001)
Batch Size	256
Hardware	CUDA GPU

TRAINING RESULTS

Chronological Split: 80% Train (~2.2M) / 20% Test (~550k). Never shuffle time-series data.

Epoch	Train Loss (MSE)	Val Loss (MSE)	Train RMSE	Val RMSE
1	0.8399	1.0052	0.92	1
2	0.6631	1.0293	0.81	1.01
3	0.5754	0.982	0.76	0.99
4	0.5297	1.0882	0.73	1.04
5	0.5009	0.998	0.71	1

The Overfitting Gap: Training loss continues dropping, but Validation loss plateaus around 1.0.

WHAT I CAN IMPROVE

What I Did	Why I Did It
Fixed Data Split (70/15/15)	Created a true validation set to monitor unseen data during training.
Added Dropout (0.3)	Forced the network to rely on varied patterns, reducing the overfitting gap.
Early Stopping (Patience=5)	Automatically halts training at the best validation loss before overfitting worsens.

IMPROVEMENT RESULTS

	Before	**After**
Epochs ran	5 (fixed)	14 (auto stopped)
Train Loss	0.5009	0.4967
Val Loss	0.9980	0.8479
Train RMSE	0.71	0.70
Val RMSE	1.00	0.92
Overfitting gap	0.50	0.35

CONCLUSION

Project Success: Successfully built an end-to-end time-series pipeline, processing 19.3M raw rows into a predictive model.

Key Takeaways:

- **Big Data Requires Big Tools:** PySpark effortlessly handled parallel processing where standard libraries would crash.
- **Time-Series Rules are Strict:** Chronological splitting and sequential sliding windows are mandatory for accurate forecasting.
- **Preprocessing is Everything:** Model intelligence relies entirely on the quality of feature selection and consistent scaling.

Final Thought: I merged Data Engineering and Machine Learning to transform massive, raw meteorological files into a system capable of predicting tomorrow's drought severity today.